LEAP2 Cluster User Guide



Contents

SPECIAL NOTICE ON FILESYSTEMS AND USAGE LIMITS

Running Jobs

- Partitions
  - Partitions, resources and limits
- Working with environment modules
- MPI job compilation
  - Sample hello_mpi.c code
  - Sample environment modules use at code compilation
- SLURM batch directives
  - Common batch directives
- Fundamental resource requests
  - Wall time
  - Nodes, tasks and CPUs
  - Memory
  - Partition
- Typical job types
  - Workflows and intended partitions
  - MPI
    - MPI batch script
  - Shared memory
    - OpenMP compiler flags
    - Shared memory (OpenMP) or multi-threaded batch script
  - Serial
    - Serial batch script
  - Interactive
    - Interactive command-line submission examples
- Extended job specifications and capabilities
  - Job arrays
  - Jobs with dependencies
  - PBS compatibility wrappers and nodefile script
    - PBS_NODEFILE batch script example
- Interacting with the job scheduler
  - Common commands

sbatch submission example
squeue example
scontrol example (show job details)
scancel example
Additional Resources

SPECIAL NOTICE ON FILESYSTEMS AND USAGE LIMITS

All users should actively work to minimize data consumption on the cluster by **maintaining a maximum data use of 1TB in user home directories and any other shared filesystems to which the user may have access**. It is the user's responsibility to stay within these limits and ensure they are not using excessive disk space. Use of excessive disk space has the potential of affecting all users of the system and must be avoided. **Use of excessive disk space (in excess of 1TB in a user's home directory or other shared filesystem) may result in data removal and suspension of a user's account.** Logging into the system constitutes acknowledgement of this requirement.

Running Jobs

Job scheduling for the LEAP2 cluster is handled by SLURM (Simple Linux Utility for Resource Management). SLURM provides the interface to allow users to request job resources on the various nodes in the cluster, and to specify job attributes and constraints. This allows SLURM to allocate resources and classify jobs to enable fair scheduling and high throughput. Login nodes are reserved for establishing ssh connections to the cluster. These nodes provide shared access to the cluster and serve as the submission point for jobs. Due to the shared nature of the cluster resources, login nodes should not be used for interactive jobs and applications, instead, when interactive jobs are required, they should be submitted to the scheduler using the `sinteractive` command described below.

Partitions

Under SLURM, "partitions" are a primary scheduling concept to define and present system resources. These equate to "queues" for some other job scheduling systems. There are six partitions currently available on the LEAP2 cluster - "parallel", "shared", "himem", and "longrun", "gpu1" and "gpu2". Each partition is defined to

provide access to specific resource types (i.e. compute, himem, gpu nodes), coordinate specific workflows (i.e. parallel/serial workflows) or provide extended node use (i.e. longrun). Consult the table below for SLURM partitions, resources and limits when formulating resource requests to the LEAP2 cluster.

Partitions, resources and limits

Partition	Node type	Job default/max cores per node	Job default/max execution time	Job default/max mem per node	Intended use	Backfilling	Access
parallel	48 CPU cores 256GB RAM 350GB SSD	48/48	1 hour / 14 days	256GB / 256GB	Parallel (distributed memory)	No, exclusive	All users
shared	48 CPU cores 256GB RAM 350GB SSD	1/48	1 hour / 48 days	5GB / 256GB	Serial, parallel (shared memory)	Yes, shared	All users
himem	48 CPU cores 1.5TB RAM 800GB SSD	1/48	1 hour / 28 days	32GB / 1.5TB	Serial, parallel (shared memory)	Yes, shared	All users
longrun	48 CPU cores 256GB RAM 350GB SSD	1/48	1 hour / 60 days	5GB / 256GB	Serial, parallel (shared or distributed memory)	user specifies	Special request
gpu1	48 CPU cores 256GB RAM 800GB SSD	1/48	1 hour / 14 days	5GB / 256GB	Serial, parallel (shared or distributed memory)	user specifies	All users
gpu2	48 CPU cores 256GB RAM 800GB SSD	1/48	1 hour / 14 days	5GB / 256GB	Serial, parallel (shared or distributed memory)	user specifies	All users

Working with environment modules

Environment modules on the LEAP2 cluster provide the ability to conveniently switch pathing for libraries between versions or varied implementations of system software in an effort to help avoid the use of un-expected versions of tools or mismatches in tools used at compilation and runtime. Where there are multiple versions or

differing implementations of system software, often there will be an environment module defined for the additional application on the system.

The primary commands used to manage environment modules are as follows:



- **`module avail`** - show available modules on the system
- **`module list`** - show modules currently activated
- **`module load <module name>`** - activate the module with <module name>
- **`module unload <module name>`** - de-activate the module with <module name>

As a example, environment modules are used during compilation of the "hello_mpi.c" parallel hello world program shown below, and then executed on cluster compute nodes via an MPI batch submission script that also uses the environment module.

MPI job compilation

The Message Passing Interface, or MPI, allows the user to leverage LEAP2's greatest advantage over high-end workstations, enabling the parallelization of executing codes across multiple processors and nodes and scaling resources far beyond the most powerful workstations. Combined with LEAP2's high-speed interconnect with minimal latency, LEAP2 compute resources in effect become a single system with far greater numbers of processors and much more physical memory. In order to run an MPI-capable code, MPI libraries are required at compilation and execution time.

As an example for MPI users, included here is a sample MPI-capable "hello world" program and steps for compilation under openmpi_4.1.5. Start by saving the below code to a file named "hello_mpi.c" in your current directory on the LEAP2 cluster.

Sample hello_mpi.c code

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {

   int  numtasks, taskid, len;
   char hostname[MPI_MAX_PROCESSOR_NAME];

   // Initialize the MPI environment
   MPI_Init(&argc, &argv);

   // Get the number of processes
   MPI_Comm_size(MPI_COMM_WORLD, &numtasks);

   // Get the rank of the process
   MPI_Comm_rank(MPI_COMM_WORLD, &taskid);

   // Get the hostname
   MPI_Get_processor_name(hostname, &len);

   // Print hello world message
```

```
    printf("Hello world from machine %s, rank %d out of %d processors\n", hostname, taskid, numtasks);

    // Finalize the MPI environment
    MPI_Finalize();
}
```

Now use the environment modules system to load any necessary environment settings to ensure you are using the tools you expect. In this case, we want to demonstrate using the OpenMPI-4.1.5 MPI libraries and compiler for compilation of the code.

Sample environment modules use at code compilation

```
[maxpwr@leap2 mpi]$ module list
Currently Loaded Modulefiles:
 1) gcc/11.2.0   2) slurm/slurm/21.08.8
[maxpwr@leap2 mpi]$ module avail
------------------------------------------------------------------ /cm/local/modulefiles ----------------------
-------------------------------------------
boost/1.77.0            cm-bios-tools  cuda-dcgm/3.1.3.1  gcc/11.2.0       luajit        module-info
openmpi/mlnx/gcc/64/4.1.5rc2  shared
cluster-tools-dell/9.2  cmd            dot                ipmitool/1.8.18  mariadb-libs  null         python3
slurm/slurm/21.08.8
cluster-tools/9.2       cmjob          freeipmi/1.6.8     lua/5.4.4        module-git    openldap
python39
...

[maxpwr@leap2 mpi]$ module load openmpi/mlnx/gcc/64/4.1.5rc2
[maxpwr@leap2 mpi]$ which mpicc
/usr/mpi/gcc/openmpi-4.1.5rc2/bin/mpicc
[maxpwr@leap2 mpi]$ mpicc -o openmpi-4.1.5_hello_mpi hello_mpi.c
```

If no errors were received from the last step shown above (compilation using mpicc) there should now be an MPI compiled parallel hello world executable named "openmpi-4.1.5_hello_mpi" in your current directory. The compiled executable can now be submitted to the LEAP2 cluster job scheduler using SLURM-style directives within an MPI batch script . The primary submission methods (srun, salloc and sbatch) can be used though this guide focuses on the use of sbatch submission scripts, as they provide many advantages over other methods. The SchedMD SLURM documentation (https://slurm.schedmd.com/) can be consulted for using methods other than sbatch.

SLURM batch directives

SLURM-style batch directives, or what other scheduling systems might refer to as job "limits", are constraints specified by the user at submission time to request resources (the most critical of these being the requested partition, runtime, number of processor cores/nodes/tasks, and potentially memory, depending on the job's requirements).

Common batch directives

Description	Specification/directive	Short equivalence or example
Scripted directive	`#SBATCH`	`#SBATCH`
Job name	`#SBATCH --job-name=my_job`	`#SBATCH -J my_job`
Requested wall-clock time limit	`#SBATCH --time=24:0:0`	`#SBATCH -t <min>` `#SBATCH -t <days-hh:min:sec>`
Partition in which to run the job	`#SBATCH --partition=<name>`	`#SBATCH -p <name>`
Number of nodes requested	`#SBATCH --nodes=1`	`#SBATCH -N min-max`
Number of cores per node	`#SBATCH --ntasks-per-node=28`	
Memory required per node	`#SBATCH --mem=<MB>`	`Ex. #SBATCH --mem=4096`
Memory required per allocated CPU	`#SBATCH --mem-per-cpu=<MB>`	`Ex. #SBATCH --mem-per-cpu=4096`
Send e-mail at the end of the job	`#SBATCH --mail-type=end`	
User's e-mail address	`#SBATCH --mail-user=netid@txstate.edu`	
Account under which to run	`#SBATCH --account=<my_account>`	`#SBATCH -A <my_account>`
Working directory	`#SBATCH --workdir=<dir-name>`	
Share nodes	`#SBATCH --shared`	
Dedicated nodes	`#SBATCH --exclusive`	
Job arrays	`#SBATCH --array=<array_spec>`	`Ex. (sbatch only) #SBATCH --array=1-10`
Job dependency	`#SBATCH --depend=<state:job_id>`	
Request specific node(s)	`#SBATCH --nodelist=<nodes>`	`Ex. #SBATCH --nodelist=himem-002`

Fundamental resource requests

While many of the above batch directives are optional, three resource requests are essential to optimal use of the cluster - partition, walltime, and tasks/nodes/CPUs required for the job. A fourth specification, memory, is not required, but can be essential in getting the best use of the cluster when sharing nodes is possible.

Wall time

The wall time (or runtime for the job) is an important specification, not only for successful completion of jobs, but for efficient backfilling of jobs via the job scheduler. For these reasons, default runtimes for jobs are specified by partition and are fairly short (1 hour). For jobs that need to run longer than one hour, it is necessary to request a longer runtime. This is accomplished using the **`--time`** or **`-t`** directives. Ideally, users should specify job wall times that are slightly more than the actual duration of time required for the job to run to normal completion. This benefits user workload and accumulative system throughput. For example, jobs that request 7 days of runtime but really only run for three will suffer from longer queue times than those that request wall times nearer to the actual required runtime.

Nodes, tasks and CPUs

To allow jobs to run on resources beyond a single entity (i.e. single CPU core or single compute node) it is necessary to specify the number of required nodes and/or tasks and/or CPUs(cores) for the job. Different workflows may dictate varying ways of accomplishing what is generally referred to as "task placement" to meet the job's requirements on the target systems. There are multiple combinations of ways using the job directives to accomplish this, possibly the most common include the use of the **`--nodes`** or **`-N`**, **`--ntasks`** or **`-n`**, potentially in conjunction with **`--ntasks-per-node`** and/or **`--cpus-per-task`**. The SchedMD SLURM documentation (https://slurm.schedmd.com/) can provide further details about these options.

Memory

Different resource allocations can be attained through memory requests alone. For example, node exclusivity for a job can be gained by requesting the whole, or near whole of the physical memory on a node, regardless of the partition to which the job is submitted. It is a best practice to profile your jobs, through several runs, to determine the typical memory usage and then request that amount at submission time through directives such as **`--mem`** or **`--mem-per-cpu`**. This is particularly good behavior when utilizing the "shared" or "himem" partitions, or when using resources in a shared fashion, as a properly formulated memory request can leave resources for other users and ensure specifying jobs have access to the full amount of requested memory when node assignment occurs.

Partition

Under the current configuration of the job scheduler, specifying a partition is required, as there is no default execution partition. As seen in the common batch directives table above, this is accomplished using the **`--partition`** or **`-p`** directives. Consult the Partitions section for more information about partition resources and limits.

Typical job types

Common cluster workflows can generally be said to be *parallel* (commonly MPI, possibly OpenMP), *embarassingly parallel*, *serial* or *interactive* in nature. *Parallel* jobs execute under either a "distributed memory" or "shared memory" paradigm. *Embarassingly parallel* jobs essentially launch multiple independent copies of executables that run simultaneously, but do not perform interprocess communication. *Serial* and *interactive* jobs generally run one instance of the application or executable, but may benefit from allocations of additional CPUs if the executable is multi-threaded. See the Workflows and intended partitions section for more information and the following job script examples. These seek to provide a framework for formulating your own job scripts that fit your workflow needs. The scripts can be used as a starting point, many examples are available on the Web, and your application FAQs and getting started documents may provide examples that can be modified to work on the LEAP2 cluster SLURM scheduler.

Workflows and intended partitions

The partitions (or queues) are specified to facilitate differing job types and provide specific default behaviors:

- **parallel** - The "parallel" partition should be used for "distributed memory" parallel jobs - that is, jobs that run on multiple compute nodes with inter-task communication (SPMD (https://en.wikipedia.org/wiki/SPMD), such as MPI (https://en.wikipedia.org/wiki/Message_Passing_Interface),) and applications that scale beyond the resources available on a single compute node and provide their own methods for inter-task communication. The primary distinction with the "parallel" partition is that jobs are assigned complete nodes by default - meaning single-job, single-user exclusivity of all nodes allocated to the job. This is generally considered optimal for "distributed memory" parallel work.
- **shared** - The "shared" partition should be used for "shared memory" parallel jobs - that is, multithreaded jobs that run on a single compute node (i.e. OpenMP (https://en.wikipedia.org/wiki/OpenMP) or pthreads (https://en.wikipedia.org/wiki/POSIX_Threads)). Likewise, so-called "embarassingly parallel", "serial" and interactive jobs should be run in the "shared" partition.
- **himem** - The "himem" partition should be requested for all jobs seeking to use resources on the high-memory nodes.
- **longrun** - The special access "longrun" partition is a subset of compute nodes that can be used for workflows that require extended runtimes.
- **gpu1** - The "gpu1" partition should be requested for jobs seeking to use resources on the single GPU nodes.
- **gpu2** - The "gpu2" partition should be requested for jobs seeking to use resources on the dual GPU nodes.

MPI

As mentioned previously, while MPI is likely the most common way that cluster users will enable parallelization of their codes and applications, differing implementations of MPI (OpenMPI, IntelMPI) provide alternatives, and some application workflows may provide their own methods.

MPI batch script

The below batch script correlates with the Sample hello_mpi.c code example MPI code and compilation of the "hello_mpi.c" executable in the Sample environment modules use at code compilation steps previously detailed. Note the specification of the partition "parallel" which is the preferred partition for MPI jobs. Note also that the

batch script loads the "openmpi-4.1.5" environment module that was also used at code compilation time to point to the appropriate libraries when the mpirun is executed. Save the batch script below to a file named "mpi_test.sh" to continue with the example practice.

```
#!/bin/bash -l

#SBATCH --job-name=mpi_test
#SBATCH --partition=parallel
#SBATCH -n 48
#SBATCH --time=00:05:00

echo "Starting at `date`"
echo "Running on hosts: $SLURM_NODELIST"
echo "Running on $SLURM_NNODES nodes."
echo "Running on $SLURM_NPROCS processors."
echo "Current working directory is `pwd`"

# setup modules
. /etc/profile.d/modules.sh

# Load MPI environment
module load openmpi/mlnx/gcc/64/4.1.5rc2

mpirun -np $SLURM_NPROCS ./openmpi-4.1.5_hello_mpi

echo "Program finished with exit code $? at: `date`"
```

With the above batch script saved with the file name "mpi_test.sh", you can now submit the job to the scheduler using the **`sbatch`** command as shown in the sbatch submission example below.

Shared memory

Shared memory parallelism, that is, jobs that use methods such as OpenMP, can take advantage of the full memory and CPUs on an individual node. Note that OpenMP executables must be compiled with the appropriate compiler flag.

OpenMP compiler flags

Compiler	Flag	Example compile
Intel	-openmp	**`icc -openmp -o hello_omp hello_omp.c`**
GCC	-fopenmp	**`gcc -fopenmp -o hello_omp hello_omp.c`**

Shared memory (OpenMP) or multi-threaded batch script

As an implementation of multi-threading, OpenMP can provide parallelism under the shared-memory model, which can utilize up to the full resources of a node (48 cores for compute nodes). The sample below shows how a sample OpenMP job (or multi-threaded job) might request 12 CPU cores, which would correlate with the threads launched during execution. You can set the value of the OMP_NUM_THREADS environment variable (if needed) at execution of your OpenMP-compiled application by adding **`export OMP_NUM_THREADS=<number of threads>`** with the set value corresponding to the number of tasks (**`#SBATCH -n`**) requested at submission time.

```
#!/bin/bash -l

#SBATCH -p shared
#SBATCH -n 12
#SBATCH -t 02:00:00
#SBATCH -J omp_mt_job

# setup modules
. /etc/profile.d/modules.sh

# set OMP_NUM_THREADS if needed by the executable
# by uncommenting the below line
#export OMP_NUM_THREADS=12

# load the appropriate compiler libraries if your
# executable was compiled with something other than
# the default... Ex. intel/14.0.2
#module load intel/14.0.2

./my_omp_or_mt_app
```

Serial

Jobs are generally considered "serial" if the executable is not parallelized (as compiled under OpenMPI or OpenMP, for example) or not otherwise executable using multiple CPU cores or nodes. In this case, jobs request a single task - **`#SBATCH -n 1`**, and should also request the "shared" partition along with any large memory requirements (if greater than the default is needed) to allow backfilling to available resources.

Serial batch script

```
#!/bin/bash -l

#SBATCH -p shared
#SBATCH -n 1
#SBATCH -t 02:00:00
#SBATCH -J my_serial_job

./serial_job
```

Interactive

Interactive job capabilities are designed to support users who need to run interactive command-line or graphical applications on a compute resource. Note that in order to run graphical applications and have the display exported back to the origination point, the user must initiate their session on LEAP2 from a host that is graphically enabled. Using **`ssh -X <userid>@leap2.txstate.edu`** in its most general form, should carry the X credentials of the originating session and allow X11 forwarding for the job submission using the **`sinteractive`** command. Note that sinteractive jobs should typically use the "shared" or "himem" partitions (depending on resource needs) because application runs of this sort are generally serial or multi-threaded in nature, but not parallel.

Interactive command-line submission examples

The sinteractive command to submit an interactive job will interpret many of the SLURM directives. Below are a few examples:

Request an interactive session attached to one node / one CPU (by default when not specified) in the shared partition and allocate 5GB (5120 MB) memory for the job:

```
[maxpwr@leap2 ~]$ sinteractive -p shared --mem=5120
```

Request an interactive session allocating 24 cores (in the shared partition) and 120GB (24 cores x 5120 MB) memory for the job:

```
[maxpwr@leap2 ~]$ sinteractive -p shared -n 24 --mem-per-cpu=5120
```

Extended job specifications and capabilities

Job arrays

A job array can be specified in a script when submitted using sbatch. For example,

```
[maxpwr@leap2 ~]$ sbatch --array=0-15%4 batch.sh
```

would submit batch.sh 16 times, with id's 0 through 15. The %4 is optional and would only allow 4 jobs to run concurrently. Within batch.sh, there are three environment variables that can be used: **`$SLURM_JOBID`** is sequential for each job and depends on the queue; **`$SLURM_ARRAY_JOB_ID`** is the same for all jobs in the array and equal to the **`$SLURM_JOBID`** of the first job; and **`$SLURM_ARRAY_TASK_ID`** is equal to the index specified with the array option (which could be for example **`--array=1,3,5,7`** or **`--array=1-7:2`** where 2 is the step size).

Jobs with dependencies

A job can be given the constraint that it only starts after another job has finished. In the following example, we have two Jobs, A and B. We want Job B to start after Job A has successfully completed. First we start Job A by submitting it via sbatch:

```
[maxpwr@leap2 ~]$ sbatch <jobA.sh>
```

Making note of the assigned job ID for Job A, we then submit Job B with the added condition that it only starts after Job A has successfully completed:

```
[maxpwr@leap2 ~]$ sbatch --dependency=afterok:<jobID_A> jobB.sh
```

If we want Job B to start after several other Jobs have completed, we can specify additional jobs, using a ':' as a delimiter:

```
[maxpwr@leap2 ~]$ sbatch --dependency=afterok:<jobID_A:jobID_C:jobID_D> jobB.sh
```

We can also tell slurm to run Job B, even if Job A fails, like so:

```
[maxpwr@leap2 ~]$ sbatch --dependency=afterany:<jobID_A> jobB.sh
```

PBS compatibility wrappers and nodefile script

As seen in the MPI batch script example previously, there are a number of important environment variables set at job submission time, one of which is **$SLURM_NODELIST**, which produces a node list in a regular expression format that may not be what the application is expecting. In these cases, it may be possible to use the PBS compatibility wrappers resident in SLURM to provide the job hostlist in a PBS-style (one hostname per line) format. The below batch script demonstrates one possible way this could be done to run an MPI-compiled hello world program (with the assumption the example executable "openmpi-4.1.5_hello_mpi" needed the PBS-style node file).

PBS_NODEFILE batch script example

```
#!/bin/bash -l

#SBATCH -p  parallel
#SBATCH -n 48
#SBATCH -t 00:02:00
#SBATCH --mem-per-cpu=5120

# We want a list of nodes, one per line.
# Use the PBS compatibility wrapper to make a PBS-style nodefile
PBS_NODEFILE=`/usr/bin/generate_pbs_nodefile`

/bin/cat $PBS_NODEFILE

# How many cores total do we have?
NO_OF_CORES=`cat $PBS_NODEFILE | egrep -v '^#'\|'^$' | wc -l | awk '{print $1}'`

# setup modules
. /etc/profile.d/modules.sh

# Load MPI environment
module load openmpi/mlnx/gcc/64/4.1.5rc2

mpirun -np $NO_OF_CORES -machinefile $PBS_NODEFILE ./openmpi-4.1.5_hello_mpi
```

Interacting with the job scheduler

Users interact with the SLURM workload manager, primarily utilizing a small set of commands to submit jobs to the system, potentially checking job statuses and sometimes modifying or canceling the submitted jobs.

Common commands

Command	Description
`sacct`	report job accounting information for active or completed jobs
`salloc`	allocate resources for a job in real time
`sbatch`	submit a job script for later execution
`scancel`	cancel a pending or running job
`scontrol`	provides various capabilities based on specification
`sinfo`	reports configured partitions, availability, attributes, allocated nodes and state
`sinteractive`	submit an interactive job request
`squeue`	reports the state of jobs and, by default, reports the running jobs in priority order followed by the pending jobs in priority order
`srun`	used to submit a job for execution in real time

sbatch submission example

```
[maxpwr@leap2 mpi]$ sbatch mpi_test.sh
Submitted batch job 11228
```

squeue example

```
[maxpwr@leap2 mpi]$ squeue
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
             11228  parallel mpi_test   maxpwr  R       0:05      1 compute-108
```

scontrol example (show job details)

```
[maxpwr@leap2 mpi]$ scontrol show job 11228
...
extended job details
...
```

scancel example

```
[maxpwr@leap2 mpi]$ scancel 11228
```

Additional Resources

A page specific to Grid Engine to SLURM conversions is here -> Grid Engine to SLURM Conversion
A SchedMD-provided two page command options summary is here -> SLURM two-page Command Options Summary (https://slurm.schedmd.com/pdfs/summary.pdf)
A SchedMD-provided conversion table for popular job schedulers is here -> SLURM Conversions Rosetta Stone (https://slurm.schedmd.com/rosetta.html)
The SchedMD SLURM documentation can be found here -> SchedMD SLURM documentation (https://slurm.schedmd.com)
The SchedMD quick start user guide is here -> SLURM Quick Start User Guide (https://slurm.schedmd.com/quickstart.html)

[1] (http://itrcstats.itrc.txstate.edu/wiki)

Retrieved from "https://itrcstats2.itrc.txstate.edu/wiki/index.php?title=LEAP2_Cluster_User_Guide&oldid=69"

This page was last edited on 23 October 2023, at 18:01.